

Lesson 10: State Management

Learning Objectives

- Understand the Terraform state file structure
 - Inspect state with CLI commands
 - Manipulate state: list, show, mv, rm, import
 - Understand remote state concepts
 - Handle state locking
-

1. What is Terraform State?

The **state file** (`terraform.tfstate`) is a JSON file that maps your Terraform configuration to real-world resources. It is Terraform's **source of truth** for what exists.

```
{
  "version": 4,
  "terraform_version": "1.7.0",
  "resources": [
    {
      "module": "root",
      "mode": "managed",
      "type": "aws_s3_bucket",
      "name": "main",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "id": "my-bucket",
            "arn": "arn:aws:s3:::my-bucket",
            "bucket": "my-bucket",
            "region": "us-east-1",
            "tags": {
              "Environment": "dev"
            }
          }
        }
      ]
    }
  ]
}
```

What state stores:

Data	Example
Resource identity	<code>id</code> , <code>arn</code>
Resource attributes	<code>bucket</code> , <code>region</code> , <code>tags</code>
Metadata	<code>terraform_version</code> , <code>schema_version</code>
Dependencies	Implicit dependency graph
Provider info	Which provider created the resource

2. State File Location

By default, the state file is stored locally:

```
terraform.tfstate      # Default file
terraform.tfstate.backup # Backup from the last state change
.terraform/
└─ terraform.state     # State version lock
```

The `terraform.tfstate.backup` File

Every time Terraform writes a new state file, it copies the **previous** state to `.backup`. This provides a safety net:

```
# If something goes wrong, restore from backup
cp terraform.tfstate.backup terraform.tfstate
```

3. Inspecting State

`terraform state list`

Lists all resources tracked in the state:

```
terraform state list
```

Output:

```
aws_s3_bucket.main
aws_dynamodb_table.main
aws_sqs_queue.main
```

Use with filtering:

```
# Filter by resource type
terraform state list 'aws_s3_bucket.*'

# Filter by module path
terraform state list 'module.storage.*'
```

terraform state show

Shows all attributes of a specific resource:

```
terraform state show aws_s3_bucket.main
```

Output includes full details:

```
# aws_s3_bucket.main:
resource "aws_s3_bucket" "main" {
  arn                  = "arn:aws:s3:::my-bucket"
  bucket               = "my-bucket"
  bucket_domain_name   = "my-bucket.s3.amazonaws.com"
  bucket_regional_domain_name = "my-bucket.s3.us-east-1.amazonaws.com"
  hosted_zone_id       = "Z3AQBSTGFYJSTF"
  id                   = "my-bucket"
  region               = "us-east-1"
  tags                 = {
    "Environment" = "dev"
  }
  tags_all              = {
    "Environment" = "dev"
  }
}
```

terraform state pull

Downloads the state from its current location (local or remote) and prints it as JSON:

```
terraform state pull
```

4. Manipulating State

terraform state rm

Removes a resource from state **without destroying** the real resource:

```
terraform state rm aws_s3_bucket.main
```

After removal, Terraform no longer manages the bucket. A subsequent `terraform plan` will show it as `+ create` (Terraform wants to create a new one).

terraform state mv

Moves a resource to a new address in the state:

```
# Rename a resource in state (matches config change)
terraform state mv aws_s3_bucket.old_name aws_s3_bucket.new_name

# Move into a module
terraform state mv aws_s3_bucket.main module.storage.aws_s3_bucket.main
```

Useful when you refactor your config and rename resources.

terraform state replace-provider

Updates the provider reference in state (rare):

```
terraform state replace-provider \
  hashicorp/aws \
  registry.example.com/hashicorp/aws
```

5. State Push and Pull

terraform state pull

Download state to stdout:

```
terraform state pull > state-backup.json
```

terraform state push

Upload a local state file to the configured backend:

```
terraform state push state-backup.json
```

Warning: Push overwrites the remote state. Use with extreme caution.

6. State in Practice: Refactoring Workflow

Scenario: Renaming a resource

1. **Edit config** — change the resource's local name:

```
# Before
resource "aws_s3_bucket" "old_name" {
  bucket = "my-bucket"
}

# After
resource "aws_s3_bucket" "new_name" {
  bucket = "my-bucket"
}
```

2. **Move in state** — tell Terraform the old state maps to the new name:

```
terraform state mv aws_s3_bucket.old_name aws_s3_bucket.new_name
```

3. **Plan** — confirms no changes needed:

```
terraform plan
# No changes. Your infrastructure matches the configuration.
```

If you skip step 2, Terraform would try to **destroy** the old resource and **create** a new one (even though it's the same bucket).

7. Common State Commands Summary

Command	Purpose
<code>terraform state list</code>	List all resources in state
<code>terraform state show <addr></code>	Show details of one resource
<code>terraform state pull</code>	Export state as JSON
<code>terraform state push</code>	Import state file (dangerous)
<code>terraform state rm <addr></code>	Remove from state (keep resource)
<code>terraform state mv <from> <to></code>	Rename in state

Command	Purpose
<code>terraform state replace-provider</code>	Change provider reference

8. Key Takeaways

- State maps config to real resources — it's Terraform's source of truth
- State is stored in `terraform.tfstate` by default (local backend)
- `.terraform.tfstate.backup` is created before every state write
- `terraform state list` and `show` are read-only and safe
- `terraform state rm` and `mv` modify state — use carefully
- After renaming a resource in config, use `terraform state mv` to avoid destroy/recreate
- Never edit `terraform.tfstate` manually — use the CLI commands instead

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer - Eyuel M.